



Solemne 3

INDICACIONES

- Esta Solemne se realiza de forma individual.
- La solemne debe ser rendida en la plataforma `http://codetrack.cl/`
- Para entrar debe escribir su correo institucional y como password su rut (solo numeros).
- Lea cada pregunta con atención y programe en la plataforma.
- No se permite el uso de dispositivos electrónicos como celulares, relojes inteligentes, tablets, etc. Quien sea sorprendido/a con alguno de ellos, se le evaluará con la nota mínima.
- Cualquier consulta sobre la plataforma debe ser resuelta con los profesores a cargo.
- Puede usar su cheat sheet impreso y entregado en clases.

Problemas

1. Limpieza y resumen de un catalogo estelar con pandas.

Enunciado

Un catalogo pequeno de estrellas fue armado desde distintas fuentes y guardado en un archivo CSV. Antes de comparar las estrellas, se debe leer la tabla, inspeccionarla, eliminar mediciones incompletas y crear una columna fisica derivada.

El archivo CSV se encuentra en la siguiente ruta:

```
1 url = (  
2     "https://raw.githubusercontent.com/jperaltac/pcfi161/"  
3     "main/Solemnes/2026/S3/catalogo_estrellas_s3.csv"  
4 )
```

Realice las siguientes operaciones:

- Importe `numpy`, `pandas` y `matplotlib.pyplot`. Lea el archivo CSV usando `pd.read_csv(url)` y guardelo en un DataFrame llamado `estrellas`.
- Inspeccione la tabla mostrando las primeras filas, dimensiones, tipos de datos y cantidad de valores faltantes por columna.
- Cree un DataFrame llamado `estrellas_limpias` eliminando solo las filas que tengan datos faltantes en `temperatura_K`, `radio_Rsol`, `distancia_pc` o `magnitud_V`. Use `copy()` al final.
- En `estrellas_limpias`, cree la columna `luminosidad_rel` usando

$$L_{\text{rel}} = R^2 \left(\frac{T}{5778} \right)^4,$$

donde R es `radio_Rsol` y T es `temperatura_K`.

- Seleccione las estrellas candidatas que cumplen simultaneamente:

$$\text{distancia_pc} < 20, \quad \text{magnitud_V} < 0.5, \quad \text{luminosidad_rel} > 5.$$

Muestre solamente `nombre`, `tipo`, `distancia_pc`, `magnitud_V` y `luminosidad_rel`.

- Agrupe `estrellas_limpias` por `tipo`. Para cada tipo, calcule cantidad de estrellas, temperatura media, distancia mediana y luminosidad relativa media. Ordene el resumen de mayor a menor luminosidad relativa media.
- Grafique `temperatura_K` versus `luminosidad_rel` usando un color distinto para cada `tipo`. Agregue etiquetas, leyenda y grilla.
- Imprima una frase que indique cual es el tipo con mayor luminosidad relativa media. La respuesta debe salir desde la tabla agrupada, no escrita a mano.

2. Ordenamiento y búsqueda de periodos orbitales.

Enunciado

Un equipo observa exoplanetas y registra sus periodos orbitales en dias:

```
periodos = [12.4, 3.1, 8.7, 25.0, 1.9, 14.2, 6.5, 18.3, 30.5, 4.8].
```

Utilice los algoritmos entregados a continuacion. No debe escribir otro algoritmo de ordenamiento ni otro algoritmo de búsqueda:

```
1 def bubble_sort(lista):
2     n = len(lista)
3     for i in range(n - 1):
4         for j in range(n - 1 - i):
5             if lista[j] > lista[j + 1]:
6                 lista[j], lista[j + 1] = lista[j + 1], lista[j]
7     return lista
8
9 def binary_search(lista_ordenada, objetivo):
10     low, high = 0, len(lista_ordenada) - 1
11
12     while low <= high:
13         mid = (low + high) // 2
14
15         if lista_ordenada[mid] == objetivo:
16             return mid
17         elif objetivo < lista_ordenada[mid]:
18             high = mid - 1
19         else:
20             low = mid + 1
21
22     return -1
```

Realice las siguientes operaciones:

- Defina la lista `periodos`.
- Ordene los periodos de menor a mayor usando `bubble_sort`. Debe ordenar una copia de la lista original, no la lista original directamente.
- Imprima la lista original y la lista ordenada. Verifique con un `print` que la lista original no cambio.
- Usando solamente la lista ordenada, determine el periodo minimo, el periodo maximo y la mediana. Como la lista tiene una cantidad par de datos, la mediana corresponde al promedio de los dos valores centrales.
- Use `binary_search` para buscar el periodo 14.2 en la lista ordenada. Imprima su posicion si fue encontrado.
- Use `binary_search` para buscar el periodo 10.0. Si no aparece, imprima un mensaje claro indicando que no fue encontrado.
- Recorra la lista ordenada y construya una nueva lista llamada `ventana_habitable` con los periodos entre 5.0 y 20.0 dias, incluyendo los extremos si aparecen.
- Imprima la lista `ventana_habitable`, su cantidad de elementos y una frase que explique por que `binary_search` solo tiene sentido despues de ordenar.

3. Ajuste cuadratico y estimacion de un umbral.

Enunciado

En una calibracion de laboratorio se mide la respuesta R de un detector para distintas dosis d . Los datos medidos son:

$$d = [0, 1, 2, 3, 4, 5, 6],$$
$$R = [1, 2, 2, 0, 4, 1, 6, 9, 10, 8, 16, 2, 22, 1].$$

Se desea construir un modelo cuadratico

$$R(d) = ad^2 + bd + c$$

y usarlo para estimar cuando la respuesta alcanza un umbral de seguridad

$$R = 12.$$

Realice las siguientes operaciones:

- (a) Importe `numpy` y `matplotlib.pyplot`. Defina los arreglos `d` y `R` usando `np.array`.
- (b) Use `np.polyfit(d, R, deg=2)` para obtener los coeficientes a , b y c .
- (c) Calcule los valores ajustados sobre los mismos datos medidos y guardelos en `R_fit`.
- (d) Calcule los residuos y el coeficiente R^2 :

$$\text{residuos} = R - R_{\text{fit}},$$

$$SS_{\text{res}} = \sum_i (R_i - R_{\text{fit},i})^2, \quad SS_{\text{tot}} = \sum_i (R_i - \bar{R})^2,$$

$$R^2 = 1 - \frac{SS_{\text{res}}}{SS_{\text{tot}}}.$$

- (e) Use `np.roots` para resolver cuando el modelo ajustado alcanza $R = 12$. Para ello debe resolver

$$ad^2 + bd + c - 12 = 0.$$

- (f) Filtre las raíces: conserve solo soluciones reales dentro del rango medido, es decir, entre 0 y 6.
- (g) Grafique los datos medidos con `plt.scatter`, el ajuste cuadrático con `plt.plot` usando una grilla fina de dosis, y la línea horizontal $R = 12$. Agregue etiquetas, leyenda y grilla.
- (h) Imprima a , b , c , R^2 y la dosis estimada para alcanzar el umbral. Agregue una frase que explique por que no basta con tomar cualquier raíz entregada por `np.roots`.